

Partial Scalar Replacement of Objects

Malay Kedia(23b1034) and Vishwaajith N.K(23b1038)

April 19, 2026

1 What are we doing?

1.1 Scalar replacement

Scalar replacement is an optimization technique where we replace an object with its individual fields and also convert any function calls with that object as the receiver to static calls. This is done to reduce the number of Heap accesses and allocations, which are expensive operations. But this optimization is a per object optimization which is only possible under very specific conditions. These conditions are

- there is no instance in which there is a local variable which may point to this object and some other object.
- there is no instance in which there is a global variable which may point to this object.
- there is no instance in which there is a field which may point to this object.
- there is no instance in which it is passed as an argument to a method which changes the state of the object.
- It does not escape the method in which it is created.

1.2 Partial scalar replacement

Partial scalar replacement is an aggressive optimization that relaxes some of the conditions of scalar replacement. The conditions that we relax are the following:

- there is no instance in which there is a global variable which may point to this object.
- It does not escape the method in which it is created.

We relax these conditions in the hope that most of the time, these conditions are not violated and we can still get the benefits of scalar replacement. And when these conditions are violated, we materialize the object and perform the necessary heap accesses and allocations. This optimization is more aggressive than scalar replacement and can give us better performance in some cases. But at the same time, if there is a lot of violations of the relaxed conditions, then we may end up with worse performance than no scalar replacement at all.

But, this optimization is not possible if there is a nontrivial constructor since we cannot guarantee that the constructor will not have any side effects. So, we only consider classes with trivial constructors for this optimization.

- 2 What analysis did we use?
- 3 How do we transform the code?
- 4 Results